Generative Repertoire Factory — Compilation, Labs and Experiment Protocol

Faculty Systems | Independent implementation pack | Working draft v0.1 | 27 October 2026

Status: proposed engineering and domain specifications; not a completed implementation or validated market claim.

Mission and separation

- Claude Code writes and tests software; a bounded Claude/model Skill may generate domain repertoire candidates. Generation never modifies the harness architecture or deploys a method.
- Inputs: Faculty objectives, Position ontology, incentives, existing Skills, Rubrics, Policies, Gates, source literature, scenarios, historical evidence and negative examples.

Candidate output schema

- Generate Position archetypes, Strategies, Playbooks, Methods, conditional Lines, Moves, composed Skill graphs, JEV Rubrics, proposed nonmandatory Gates, assumptions, failure modes, countermoves and experiment designs.
- Each candidate records source provenance, novelty claim, falsifiable mechanism, expected Position effects, applicability, costs/risks, missing Skills and negative repertoire. Never invent field success rates.

Qualification ladder

- Generated → schema-valid → statically compiled → JEV/Vichara evaluated → lab-qualified → controlled field experimental → field-qualified → active/restricted/deprecated/retired. Evidence qualification and production availability are separate dimensions.
- Historical replay, synthetic simulation, shadow mode and authorized field trials provide different evidence strength. No lab-only result becomes field-proven.

Experiments and learning

- Record baseline Position, treatment Line, alternative/comparator, selection rationale, execution fidelity, confounders, adverse effects, costs and outcome windows. Use controlled comparisons when feasible.
- Learning Engine proposes applicability refinement, Skill routing, rubric calibration or retirement. Human/domain governance approves consequential promotion and method distillation.

Claude prompt contract

- Generate candidate repertoire for the specified Faculty only. Use pinned schemas. Do not invent authority, policies, tools, agents, new runtimes or performance claims. Include at least one failure mode and falsification test per Move.
- Output only typed candidate definitions, explanation Markdown, fixture cases, negative cases and evaluation report; stop before activation.

Acceptance

Reject structurally invalid graphs and unsupported Skills; verify rubric reproducibility; test simulation hindsight leakage; ensure field qualification requires governed execution evidence; preserve rejected candidates and reasons.

Implementation rules and next approvals

Use a dedicated repository, versioned contracts, offline fixtures, architecture tests, SQL-backed ledger/outbox, blob artifact store and durable queue where required. Human review is required for schema changes, security decisions, commercial claims and production release.

Claude Code must not introduce LLM-agent frameworks, autonomous planners, meta-harnesses, unbounded model loops or undocumented runtime abstractions. All model calls are bounded Skills or evaluators behind approved interfaces.